

MATERIA: DESARROLLO WEB BACKEND

GUÍA DE LABORATORIO N° 10

Construcción de una API REST

Material de Práctica de Laboratorio

CONTENIDO DE LA GUÍA

1. Fundamentos de API REST

- Recursos, verbos HTTP, JSON y códigos de estado

2. Creación del proyecto desde cero

- Proyecto api-tareas con PostgreSQL

3. Habilitar la capa de API

- El comando install:api y routes/api.php

4. Modelo y migración del recurso

5. Validación con Form Request

6. Formato de salida con API Resources

7. CRUD RESTful con apiResource

8. Pruebas con Postman y cURL

OBJETIVO DE LA GUÍA

Construir desde cero una API REST en Laravel 13 que exponga un recurso completo mediante operaciones CRUD. Al finalizar, el estudiante será capaz de habilitar la capa de API con install:api, definir rutas con apiResource, validar la entrada con Form Requests, dar formato a la salida con API Resources, aplicar paginación y los códigos de estado correctos, y verificar todos los endpoints con Postman y cURL. Esta API quedará lista para asegurarse con Sanctum en la GL11.

Datos de la asignatura	
Asignatura	INF560 – Desarrollo Web Backend
Carrera	Ingeniería Informática · Facultad de Ciencias Puras
Universidad	Universidad Autónoma Tomás Frías (UATF) – Potosí
Docente	M. Sc. Huáscar Fedor Gonzales Guzmán
Stack	Laravel 13 · PHP 8.3+ · PostgreSQL · Laravel Sanctum
Duración estimada	2 horas (1 sesión de laboratorio)

1. Objetivos, Prerrequisitos y Resultados

Objetivos específicos

- Comprender los principios de una API REST: recursos, verbos HTTP, formato JSON y códigos de estado.
- Habilitar la capa de API de Laravel 13 con el comando `install:api` y entender qué genera.
- Construir un CRUD RESTful completo con un controlador de recurso (`apiResource`) que devuelve JSON.
- Validar la entrada con Form Requests y dar formato consistente a la salida con API Resources.
- Aplicar paginación y los códigos de estado correctos (200, 201, 204, 404, 422).
- Probar todos los endpoints con Postman y cURL.

Prerrequisitos

- Haber completado las guías previas o manejar con fluidez migraciones, modelos Eloquent, controladores y validación.
- Tener instalados PHP 8.3+, Composer y el cliente de PostgreSQL, con un servidor PostgreSQL accesible localmente.
- Conocer los verbos HTTP (GET, POST, PUT/PATCH, DELETE) y los códigos de estado HTTP más comunes.
- Tener instalado Postman (o usar cURL desde la terminal) para consumir la API.

PROYECTO DE LA GUÍA

Esta guía construye un proyecto nuevo e independiente llamado `api-tareas`. Aquí lo dejaremos como una API pública (sin autenticación) para concentrarnos en la mecánica REST. En la GL11 tomaremos este mismo proyecto y le añadiremos seguridad con Laravel Sanctum.

2. Marco Teórico

2.1 ¿Qué es una API REST?

Una API REST (Representational State Transfer) expone recursos —usuarios, tareas, productos— a través de URLs y verbos HTTP, devolviendo datos en formato JSON en lugar de páginas HTML. Cada recurso tiene una URL estable, y la acción a realizar se expresa con el verbo HTTP: GET para leer, POST para crear, PUT/PATCH para actualizar y DELETE para eliminar.

2.2 API vs. aplicación monolítica con Blade

A diferencia de las guías anteriores, donde el servidor devolvía vistas Blade renderizadas, una API no produce HTML ni mantiene «pantallas». Solo intercambia datos. Esto la hace consumible por cualquier cliente: una app móvil, un frontend en React o Vue, o incluso otro servidor. La separación entre el backend (la API) y el frontend (quien la consume) es la base de las arquitecturas modernas.

Aspecto	App monolítica (Blade)	API REST
Respuesta	HTML renderizado	JSON
Estado de sesión	Cookie de sesión	Sin estado (cada petición es independiente)
Cliente típico	Navegador	App móvil, SPA, otro servidor
Archivo de rutas	<code>routes/web.php</code>	<code>routes/api.php</code> (prefijo <code>/api</code>)

2.3 Verbos HTTP y códigos de estado

El contrato de una API REST se apoya en usar el verbo y el código de estado correctos. Estos son los más relevantes para un CRUD:

Verbo	Uso	Código de éxito típico
GET	Leer un recurso o una colección	200 OK
POST	Crear un recurso	201 Created
PUT	Actualizar un recurso	200 OK
DELETE	Eliminar un recurso	204 No Content

CÓDIGOS DE ERROR FRECUENTES

404 Not Found: el recurso solicitado no existe. 422 Unprocessable Entity: la validación falló (Laravel lo devuelve automáticamente con los errores). 401 Unauthorized y 403 Forbidden aparecen cuando hay autenticación; los veremos en la GL11.

2.4 routes/api.php y apiResource

En Laravel 11+ el archivo `routes/api.php` no existe por defecto: se crea con el comando `install:api`. Una vez creado, todas sus rutas quedan bajo el prefijo `/api`. El método `Route::apiResource()` registra de un solo golpe las cinco rutas del CRUD (sin `create` ni `edit`, porque una API no tiene formularios).

2.5 API Resources: dar forma a la salida

Devolver un modelo Eloquent directamente expone todas sus columnas. Un API Resource es una clase que transforma el modelo en el JSON que queremos exponer: elige qué campos mostrar, los renombra y les da formato (fechas, booleanos). Así desacoplamos la estructura de la base de datos de la respuesta pública de la API.

3. Desarrollo de la Práctica

Sigue las actividades en orden. Cada una incluye pasos numerados, los fragmentos de código con su ruta y un **Checkpoint** para verificar tu avance.

Actividad 1 · Crear el proyecto y configurar PostgreSQL (20 min)

1. Crea un proyecto Laravel nuevo y entra en su carpeta:

Terminal

```
composer create-project laravel/laravel api-tareas
cd api-tareas
```

2. Crea la base de datos en PostgreSQL:

psql

```
CREATE DATABASE api_tareas;
```

3. Configura la conexión en el archivo .env:

.env

```
DB_CONNECTION=pgsql
DB_HOST=127.0.0.1
DB_PORT=5432
DB_DATABASE=api_tareas
DB_USERNAME=postgres
DB_PASSWORD=tu_password
```

4. Ejecuta las migraciones base y levanta el servidor:

Terminal

```
php artisan migrate
php artisan serve
```

CHECKPOINT

Al visitar <http://127.0.0.1:8000> se muestra la página de bienvenida de Laravel y migrate corrió sin errores contra PostgreSQL.

Actividad 2 · Habilitar la capa de API (10 min)

1. Crea el archivo de rutas de la API con el comando oficial:

Terminal

```
php artisan install:api
```

Este comando crea `routes/api.php` y lo registra automáticamente bajo el prefijo `/api`. De paso instala Laravel Sanctum y crea la tabla `personal_access_tokens`. En esta guía no usaremos Sanctum todavía: lo dejaremos listo para la GL11.

UN PASO QUE PREPARA LA SIGUIENTE GUÍA

`install:api` hace dos cosas: habilita el enrutamiento de la API (lo que necesitamos ahora) e instala Sanctum (que aprovecharemos en la GL11). Por eso aparece una tabla `personal_access_tokens` que de momento permanecerá vacía.

2. Confirma que el archivo de rutas existe y revisa el listado de rutas de la API:

Terminal

```
php artisan route:list --path=api
```

CHECKPOINT

Existe el archivo routes/api.php y route:list muestra las rutas bajo el prefijo /api.

Actividad 3 · Modelo y migración del recurso (20 min)

1. Genera el modelo Tarea con su migración, factory y controlador de API:

Terminal

```
php artisan make:model Tarea -mfc --api
```

Las banderas: -m migración, -f factory, -c --api controlador de recurso sin create/edit.

2. Define las columnas en la migración (aún sin dueño: las tareas son públicas en esta guía):

database/migrations/xxxx_create_tareas_table.php

```
public function up(): void
{
    Schema::create('tareas', function (Blueprint $table) {
        $table->id();
        $table->string('titulo');
        $table->text('descripcion')->nullable();
        $table->boolean('completada')->default(false);
        $table->date('vence_el')->nullable();
        $table->timestamps();
    });
}
```

3. Configura el modelo con sus atributos asignables y casts:

app/Models/Tarea.php

```
<?php
namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Tarea extends Model
{
    protected $fillable = ['titulo', 'descripcion', 'completada', 'vence_el'];

    protected $casts = [
        'completada' => 'boolean',
        'vence_el'   => 'date',
    ];
}
```

4. Ejecuta la migración:

Terminal

```
php artisan migrate
```

CHECKPOINT

La tabla tareas existe con sus columnas y el modelo Tarea define \$fillable y los casts.

Actividad 4 · Validación con Form Request y formato con API Resource (25 min)

1. Crea un Form Request y un API Resource:

Terminal

```
php artisan make:request TareaRequest
php artisan make:resource TareaResource
```

2. Define las reglas de validación. authorize() devuelve true porque en esta guía no hay autenticación:

app/Http/Requests/TareaRequest.php

```
public function authorize(): bool
{
    return true;
}

public function rules(): array
{
    return [
        'titulo'      => 'required|string|max:255',
        'descripcion' => 'nullable|string',
        'completada'  => 'boolean',
        'vence_el'    => 'nullable|date',
    ];
}
```

3. Define la forma del JSON de salida en el Resource:

app/Http/Resources/TareaResource.php

```
public function toArray(Request $request): array
{
    return [
        'id'          => $this->id,
        'titulo'       => $this->titulo,
        'descripcion'  => $this->descripcion,
        'completada'   => $this->completada,
        'vence_el'     => $this->vence_el->toDateString(),
        'creada_el'    => $this->created_at->toIso8601String(),
    ];
}
```

¿POR QUÉ UN RESOURCE?

Sin el Resource, devolver el modelo expondría también columnas internas o con nombres distintos a los deseados. El Resource nos da control total sobre el JSON: aquí renombramos created_at a creada_el y damos formato a las fechas.

CHECKPOINT

Existen TareaRequest (con sus reglas) y TareaResource (con su método toArray).

Actividad 5 · El controlador CRUD (apiResource) (30 min)

1. Implementa el controlador. Cada método devuelve JSON usando el Resource; el route-model binding inyecta la tarea por su id y genera 404 automáticamente si no existe:

app/Http/Controllers/Api/TareaController.php

```
<?php
namespace App\Http\Controllers\Api;

use App\Http\Controllers\Controller;
use App\Http\Requests\TareaRequest;
use App\Http\Resources\TareaResource;
use App\Models\Tarea;
```

```

class TareaController extends Controller
{
    public function index()
    {
        return TareaResource::collection(
            Tarea::latest()->paginate(10)
        );
    }

    public function store(TareaRequest $request)
    {
        $tarea = Tarea::create($request->validated());
        return (new TareaResource($tarea))
            ->response()->setStatusCode(201);
    }

    public function show(Tarea $tarea)
    {
        return new TareaResource($tarea);
    }

    public function update(TareaRequest $request, Tarea $tarea)
    {
        $tarea->update($request->validated());
        return new TareaResource($tarea);
    }

    public function destroy(Tarea $tarea)
    {
        $tarea->delete();
        return response()->json(null, 204);
    }
}

```

2. Registra las rutas del recurso en routes/api.php:

routes/api.php

```

<?php
use Illuminate\Support\Facades\Route;
use App\Http\Controllers\Api\TareaController;

Route::apiResource('/tareas', TareaController::class);

```

apiResource genera estas cinco rutas:

Método y ruta	Acción	Propósito
GET /api/tareas	index	Listar tareas (paginadas)
POST /api/tareas	store	Crear una tarea
GET /api/tareas/{id}	show	Ver una tarea
PUT /api/tareas/{id}	update	Actualizar una tarea
DELETE /api/tareas/{id}	destroy	Eliminar una tarea

CHECKPOINT

route:list muestra las cinco rutas de tareas y el controlador devuelve JSON con el formato del Resource.

Actividad 6 · Probar la API con Postman y cURL (25 min)

Envía siempre la cabecera `Accept: application/json` para que los errores de validación lleguen como JSON (422) en lugar de HTML.

1. Crear una tarea: **POST** `/api/tareas`

cURL

```
curl -X POST http://127.0.0.1:8000/api/tareas \
-H "Accept: application/json" \
-d "titulo=Preparar laboratorio" \
-d "descripcion=Guia 10 API REST" \
-d "vence_el=2026-07-01"
```

Debe responder 201 con la tarea en formato JSON del Resource.

2. Listar tareas: **GET** `/api/tareas`

cURL

```
curl http://127.0.0.1:8000/api/tareas -H "Accept: application/json"
```

3. Ver una tarea inexistente para comprobar el **GET** `/api/tareas/999` → debe responder 404.
4. Enviar un POST sin el campo titulo para comprobar el error de validación → debe responder 422 con el detalle de los errores.
5. Actualizar y eliminar: prueba **PUT** `/api/tareas/1` y **DELETE** `/api/tareas/1` (este último responde 204 sin cuerpo).

SUGERENCIA EN POSTMAN

Crea una colección «api-tareas» con una petición por endpoint. Configura `Accept: application/json` a nivel de colección para no repetirlo en cada petición.

CHECKPOINT

Completaste el ciclo CRUD: crear (201), listar, ver, actualizar, eliminar (204), y comprobaste los errores 404 y 422.

4. Rúbrica de Evaluación

La práctica se evalúa sobre 100 puntos.

Criterio	Descripción	Pts
Proyecto y PostgreSQL	Proyecto api-tareas creado; migraciones corren sin errores contra PostgreSQL.	10
Capa de API habilitada	install:api ejecutado; routes/api.php creado y verificado con route:list.	10
Modelo y migración	Tabla tareas con sus columnas; modelo con \$fillable y casts correctos.	15
Validación y Resource	TareaRequest con reglas válidas; TareaResource con formato de salida adecuado.	20
Controlador CRUD	apiResource activo; los cinco métodos devuelven JSON y los códigos de estado correctos.	25
Pruebas y evidencias	Ciclo CRUD probado en Postman/cURL con capturas, incluidos los casos 404 y 422.	20

TOTAL 100

5. Entregables

- Repositorio Git del proyecto api-tareas con commits incrementales por actividad.
- Archivo README.md con las instrucciones de instalación y la lista de endpoints.
- Colección de Postman exportada (.json) con todas las peticiones probadas.
- Documento PDF con las capturas de las pruebas, incluidos los casos 404 y 422.

ETIQUETA DE ENTREGA (GIT TAG)

Al finalizar, marca la versión de tu entrega con la etiqueta **v1.0.0**. En la GL11 continuaremos sobre este mismo repositorio.

Terminal

```
git add .
git commit -m "GL10: API REST publica con Laravel"
git tag v1.0.0
git push origin main --tags
```